# 23. Review Report Template

*Standard truth-first review template for Claude and human reviewers*

## 1. Purpose

This template standardizes how a review should be written so that the report does not drift into vague praise, packaging theater, or untestable claims. It can be used after an implementation phase, a QA pass, a documentation update, or a pre-release gate review.

### Review Report Structure

| | Section | Must answer |
|---|---|---|
| Scope truth | What exactly was in scope? | What was the requested scope and |
| Evidence reviewed | What proof exists? | Which files, renders, tests, logs, scre |
| Findings | What passed and failed? | What is fully working, partially worki |
| Risk rating | How severe are gaps? | What blocks release, what can wait, |
| Ship decision | Can it ship? | Go / no-go and why, in plain languag |
| Next actions | What should happen next? | Bounded next phase with acceptanc |

## 2. Review template

### A. Review metadata

Review date, reviewer, branch or package name, requested scope, actual scope reviewed.

### B. Sources inspected

Code files, rendered docs, screenshots, logs, endpoint outputs, database evidence, live URLs, and user instructions.

### C. Scope truth

What was supposed to change, what actually changed, and what stayed untouched.

### D. Findings by area

Frontend, backend, data model, API contract, admin/ops, docs, deployment, or other relevant domains.

### E. Evidence table

For each major claim, list the proof source and whether it was directly inspected.

### F. Risk classification

Blocker, major, moderate, minor, cosmetic.

### G. Ship decision

PASS, PARTIAL, or FAIL and why.

### H. Exact next actions

Bounded steps for the next pass, including acceptance criteria.

## 3. Example evidence table schema

| Claim | Evidence inspected | Status | Comment |
|---|---|---|---|
| Primary user flow works end to end | UI screenshot + API response + state change | PASS | Observed directly |
| Schema and API contract align | Model file + migration + contract doc | PARTIAL | One enum mismatch remained |
| Admin action is auditable | Admin screen + audit log output | FAIL | No audit trail shown in evidence |

## 4. Recommended wording rules

- Write in direct plain language. Avoid adjectives that imply quality without evidence.
- Separate confirmed facts from inference.
- If something was not checked, say 'unverified' rather than implying it worked.

- Use PASS / PARTIAL / FAIL only against named acceptance gates.
- When a report is favorable, still include the strongest remaining risk.

## 5. Review depth levels

| Level | Use when | Expected effort |
|---|---|---|
| Quick truth pass | Need immediate honesty on a bounded phase | Inspect changed files plus primary evidence |
| Standard review | Need a trustworthy go/no-go recommendation | Inspect artifacts across all touched layers |
| Release gate review | Need a final ship decision | Inspect package order, evidence completeness, and highest-risk user journeys |

## 6. Review report skeleton to paste into Claude

Use the following output structure:

1. Scope requested
2. Scope actually reviewed
3. Evidence inspected
4. Confirmed working
5. Partially working
6. Unverified
7. Contradictions / drift
8. Risk rating
9. PASS / PARTIAL / FAIL
10. Exact next steps

## 7. Common anti-patterns to reject

- Summaries that say 'looks good' without naming proof.
- Reports that praise code organization but do not test user-facing behavior.
- Claims of readiness that ignore known partial states.
- Large copied logs with no interpretation.
- Checklist output that hides unresolved blockers.

## 8. Definition of a useful review

A useful review reduces uncertainty for the next decision-maker. After reading it, the reader should know what was checked, what is solid, where the risks are, and what the smallest safe next move is.